

```

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

df = pd.read_csv("/content/spam_or_not_spam.csv")

```

```

print(df.head())
print(df.columns)

```

```

                                email  label
0  date wed NUMBER aug NUMBER NUMBER NUMBER NUMB...      0
1  martin a posted tassos papadopoulos the greek ...      0
2  man threatens explosion in moscow thursday aug...      0
3  klez the virus that won t die already the most...      0
4  in adding cream to spaghetti carbonara which ...      0
Index(['email', 'label'], dtype='object')

```

```

df = df.dropna(subset=["email", "label"])
df = df.reset_index(drop=True)
df = df.dropna(subset=["email", "label"])
df = df.reset_index(drop=True)

```

```

X_text = df["email"]
y = df["label"].values

```

```

vectorizer = TfidfVectorizer(
    stop_words="english",
    max_features=1000
)

```

```

X = vectorizer.fit_transform(X_text).toarray()

```

```

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42
)

```

```

class SingleNeuronClassifier:

```

```

    def __init__(self, input_dim, lr=0.1):
        self.w = np.zeros(input_dim)

```

```

        self.b = 0
        self.lr = lr
        self.loss_history = []
        self.acc_history = []

    def sigmoid(self, z):
        return 1 / (1 + np.exp(-z))

    def forward(self, X):
        return self.sigmoid(np.dot(X, self.w) + self.b)

    def compute_loss(self, y, y_pred):
        eps = 1e-9
        return -np.mean(
            y * np.log(y_pred + eps) +
            (1 - y) * np.log(1 - y_pred + eps)
        )

    def train(self, X, y, X_val, y_val, epochs=100):
        for _ in range(epochs):
            y_pred = self.forward(X)
            error = y_pred - y

            # Gradient descent update
            self.w -= self.lr * np.dot(X.T, error) / len(y)
            self.b -= self.lr * np.mean(error)

            # Store training loss
            loss = self.compute_loss(y, y_pred)
            self.loss_history.append(loss) # Store training accuracy
        (THIS IS THE CHANGE)
            train_pred = self.predict(X)
            acc = accuracy_score(y, train_pred)
            self.acc_history.append(acc)

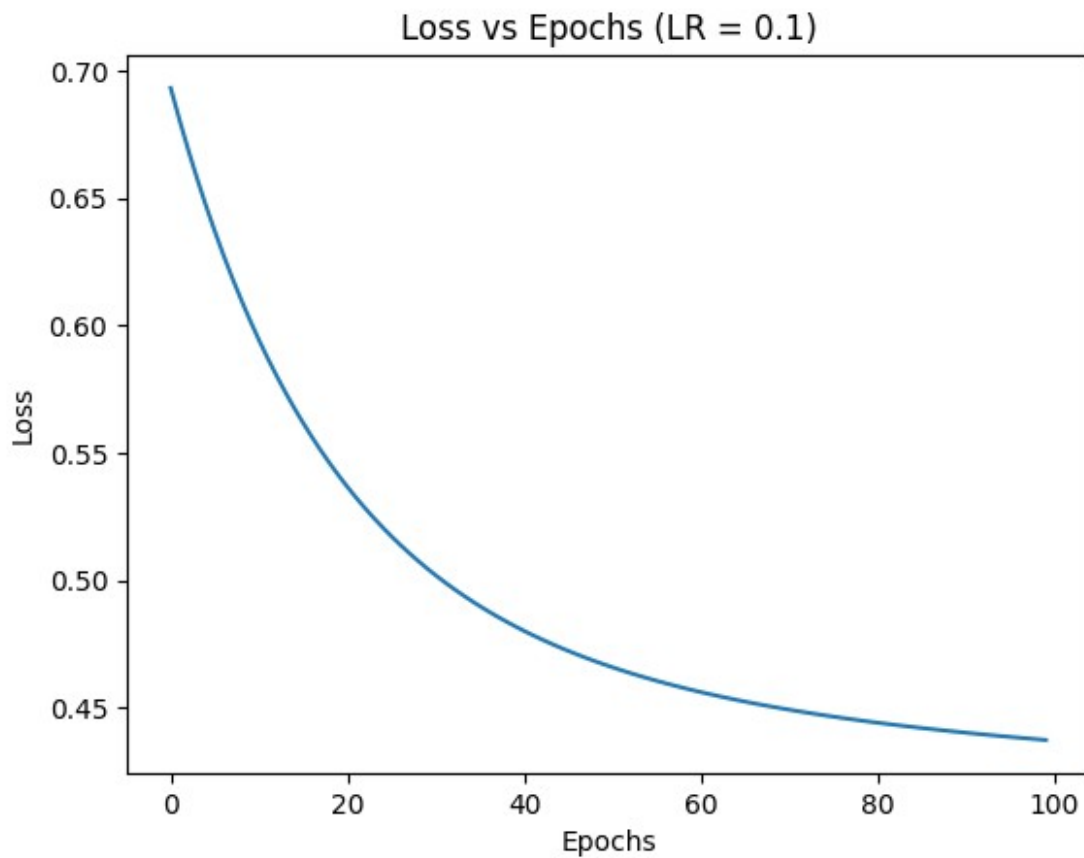
    def predict(self, X):
        return (self.forward(X) >= 0.5).astype(int)

model = SingleNeuronClassifier(
    input_dim=X_train.shape[1],
    lr=0.1
)

model.train(
    X_train, y_train,
    X_test, y_test,
    epochs=100
)

```

```
plt.figure()
plt.plot(model.loss_history)
plt.xlabel("Epochs")
plt.ylabel("Loss")
plt.title("Loss vs Epochs (LR = 0.1)")
plt.show()
```



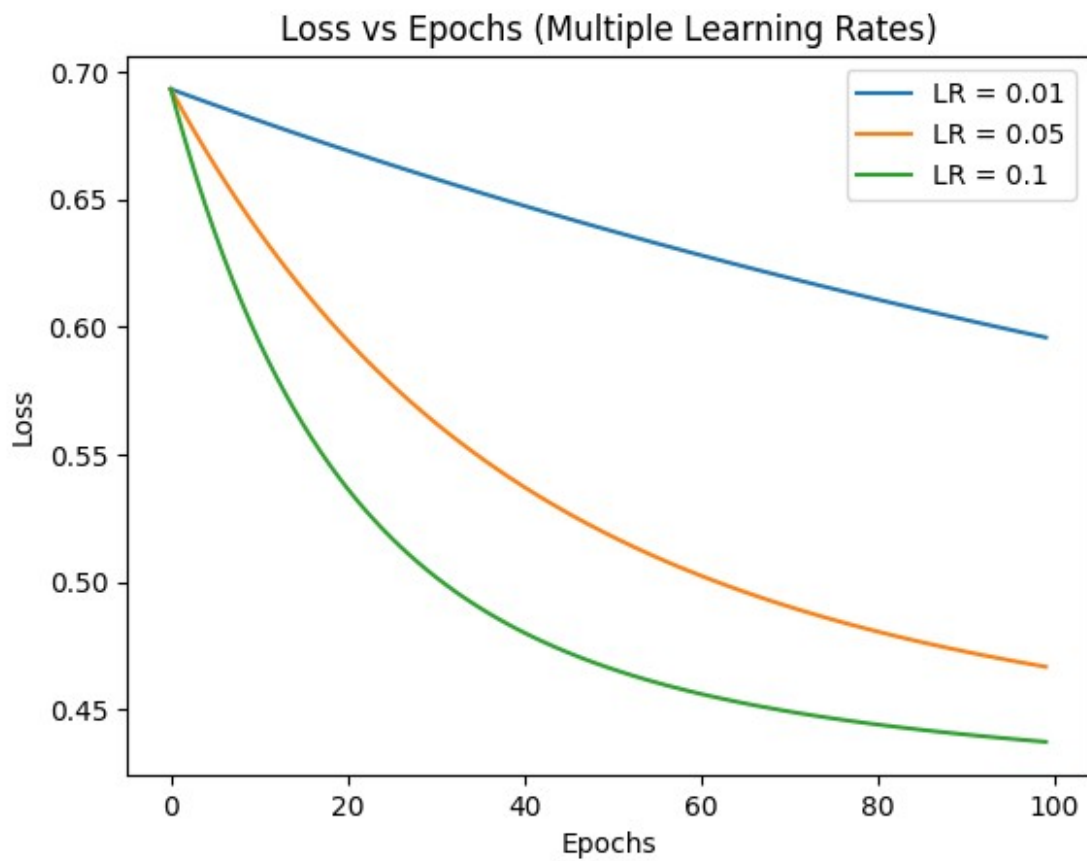
```
learning_rates = [0.01, 0.05, 0.1]
loss_curves = {}

for lr in learning_rates:
    model = SingleNeuronClassifier(
        input_dim=X_train.shape[1],
        lr=lr
    )
    model.train(
        X_train, y_train,
        X_test, y_test,
        epochs=100
    )
    loss_curves[lr] = model.loss_history
```

```
plt.figure()

for lr, loss in loss_curves.items():
    plt.plot(loss, label=f"LR = {lr}")

plt.xlabel("Epochs")
plt.ylabel("Loss")
plt.title("Loss vs Epochs (Multiple Learning Rates)")
plt.legend()
plt.show()
```



```
# Final accuracy on test data
y_test_pred = model.predict(X_test)
final_accuracy = accuracy_score(y_test, y_test_pred)

print("Final Test Accuracy:", final_accuracy)

Final Test Accuracy: 0.8333333333333334
```